# MARS: Typed Executable Hypothesis Induction for Scientific Reasoning with Small Language Models

**Anonymous submission**


## Abstract

Scientific reasoning benchmarks expose a failure mode that is easy to miss in standard agent evaluations: the difficult step is often not verifying a final answer, but constructing the right hypothesis family before the answer is known. This is especially problematic for small language models, whose extended reasoning traces can repeat shallow variants of the same wrong representation. We introduce MARS, a typed executable hypothesis-induction system that represents hypotheses as executable artifacts with typed holes, evidence contracts, measurements, validators, and residuals. A single MARSContext is shared across contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned operator promotion. The model therefore proposes local typed objects, while execution closes uncertain slots and failures are stored as structured residuals rather than as untyped bad answers. We evaluate the same kernel on DiscoveryBench-style data-driven discovery, NewtonBench-style symbolic law induction, and UltraHorizon long-horizon rule induction. With a GPT-4o-mini core, MARS reaches 40.5 HMS and 60.5 consistency-HMS on a 30-task DiscoveryBench hard-bucket slice, and 20.4 HMS / 27.1 consistency-HMS on a 239-task DB-Real audit after residual-driven promotion of original–replication study-design operators. On this audit, MARS exceeds published non-oracle GPT-4o ReAct and GPT-4-preview CodeGen references, while remaining below oracle-feedback Reflexion, which receives gold-derived revision signals. On NewtonBench it reaches 41.7% all-task and 83.3% answered-task symbolic accuracy on 24 hard-law tasks, and on UltraHorizon it scores 49.2/45.8 on all-environment easy/hard audits while recovering 5/5 exact programs in a checked sequence slice. A residual-operator audit further improves sequence recovery from 2/5 to 5/5, repairs a near-correct power law to exact recovery, and raises the most difficult DiscoveryBench raw meta-regression subset from 0.7 to 24.2 HMS. These experiments support a narrow structural claim: externalizing hypothesis construction into typed executable state localizes small-model failures and makes missing hypothesis operators measurable. Broader operator coverage and semantic rendering remain open.


This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

## Introduction

Scientific discovery tasks ask an agent to do more than answer a question. The agent must decide what should be measured, what kind of relation is plausible, which variables play stable roles, and when a proposed explanation has been falsified. Recent benchmarks make this distinction explicit. DiscoveryBench casts data-driven discovery as hypothesis search over real datasets and metadata (Majumder et al. 2024). NewtonBench evaluates interactive scientific law discovery rather than static curve fitting (Zheng et al. 2025). UltraHorizon stresses long-horizon partially observable exploration (Luo et al. 2025), while ScienceAgentBench evaluates whether agents can generate executable scientific workflows (Chen et al. 2025). Across these settings, the bottleneck is often not final verification but the construction of a useful hypothesis space before the answer is known.

This bottleneck is especially sharp for small language models. Standard inference-time scaffolds extend the trajectory: ReAct interleaves reasoning and actions (Yao et al. 2023a), Reflexion and Self-Refine add verbal feedback (Shinn et al. 2023; Madaan et al. 2023), Tree-of-Thoughts and LATS search over reasoning branches (Yao et al. 2023b; Zhou et al. 2024), and CodeAct uses executable code as the action interface (Wang et al. 2024). These methods improve many agent tasks, but they still largely rely on the model to propose the intermediate concepts that make the search fruitful. If the model never represents the task in the right coordinates, more attempts can simply explore a larger set of wrong programs.

MARS starts from a different premise: hypothesis generation should be treated as an external, executable induction problem, not as a longer natural-language trace. A scientific hypothesis is represented as an artifact with typed holes, evidence contracts, executable measurements, validation tests, and residuals. The language model proposes local moves, but the environment closes uncertain parts whenever possible. For example, instead of asking the model to guess a complete law or discovery statement, the system asks which roles must be filled, which measurement can identify each role, which residual remains after execution, and which operator would make that residual simpler on future tasks.

The central object in MARS is a shared hypothesis state, MARSContext. It is read and written by heterogeneous components: a universal hypothesis kernel, evidence-

contract compiler, estimand synthesizer, role canonicalizer, coordinate-probe engine, metamorphic verifier, and residual compiler. This is not a transcript memory. It is a typed state over hypotheses: what has been measured, which slots have been closed, which answer forms are admissible, which residuals recur, and which executable operators compress previous failures. In this respect MARS is closer in spirit to library-learning systems such as DreamCoder (Ellis et al. 2021) than to a pure agent loop, but its learned unit is not a domain-specific program alone. It is a reusable way of turning an underspecified scientific task into measurable hypothesis objects. In ReAct-style agents, a failed tool result remains contextual feedback to the model; in MARS, it is mapped to a typed residual whose type constrains the next admissible executable operators.

The contribution is threefold. First, we formulate small-model scientific reasoning as *typed executable hypothesis induction*: a sequence of typed measurements, validations, and residual-conditioned language edits. Second, we instantiate this formulation in one shared kernel that spans tabular discovery, symbolic law induction, and long-horizon rule discovery without benchmark- specific solver modules. Third, we report controlled small/large-model slices, a DiscoveryBench non-oracle/oracle comparison, and a residual-operator audit across three benchmark families. The empirical claim is deliberately narrower than "small models solve science": externalizing hypothesis generation makes failures more local and more diagnosable. The remaining gap becomes operator coverage, full-protocol scale, and rendering alignment rather than unconstrained chain-of-thought quality.

## Preliminaries

**Task model.** We consider a scientific reasoning task as an interface $x$ and an answer space $\mathcal{A}$. The interface may expose a dataset with metadata, a simulator that can be probed, a partially observable environment, or a code workspace. The answer may be a natural-language hypothesis, a symbolic law, a transition program, or a Python workflow. Each benchmark provides its own evaluator $S_x : \mathcal{A} \to \mathbb{R}$, but the system must operate before seeing the gold answer and cannot assume a benchmark-specific answer template.

**Hypothesis families.** Let $\mathcal{H}$ be a family of executable hypothesis artifacts. A hypothesis $h \in \mathcal{H}$ is not a string alone but a tuple

$$h = (z, \theta, m, v, r), \tag{1}$$

where $z$ is a typed sketch, $\theta$ is a partial assignment to its holes, $m$ is an executable measurement or transition program, $v$ is a verifier, and $r$ renders verified evidence into the benchmark answer format. This separates three operations that are often collapsed in an LLM prompt: choosing a form, measuring its slots, and writing the final answer.

**Evidence contracts.** Before proposing a final answer, MARS compiles the interface into an evidence contract

$$C(x) = \{s_i = (\tau_i, q_i, \nu_i)\}_{i=1}^{k}. \tag{2}$$

Here $\tau_i$ is the slot type, $q_i$ is the observation or computation that could close the slot, and $\nu_i$ is a local validity predicate. A contract for tabular discovery may require population, outcome, predictor, direction, and statistic slots. A contract for law induction may require coordinates, dimensionless groups, constants, and residual checks. A contract for long-horizon rule discovery may require state variables, transition clauses, and trace-level invariants.

Table 1: Posterior terms used by the shared hypothesis kernel.

| Term | Definition | Range | Weight |
|---|---|---|---|
| $E$ | executable fit or local score | $[0, 1]$ | 24 |
| Cover | contract slots covered | $[0, 1]$ | 34 |
| $V$ | metamorphic validation loss | $[0, 1]$ | 32 |
| $K$ | operator complexity | $\geq 0$ | 2 |

**Residuals and posterior scoring.** Executing $h$ on $x$ produces evidence $E(h, x)$ and a typed residual

$$\rho(h, C) = C - \mathrm{Cover}(E(h, x), C). \tag{3}$$

The kernel scores candidates by evidence fit, contract coverage, validation loss, and complexity:

$$\pi(h \mid x) \propto \exp\{\lambda_E E + \lambda_C \,\mathrm{Cover} - \lambda_V V - \lambda_K K\}. \tag{4}$$

This resembles Bayesian model selection in spirit: a useful hypothesis should explain observations while remaining compact and stable under checks. The important operational point is that $\rho$ is typed. A failure is not just a low score; it identifies a missing role, wrong coordinate system, invalid answer form, or unmodeled transition.

Table 1 makes the scoring parameterization explicit. The same decomposition is used across interfaces, while each operator normalizes its executable evidence to a comparable range before entering the shared posterior.

**Evaluation metrics.** DiscoveryBench reports Hypothesis Matching Score (HMS), a facet-based score over context, variables, and relations. NewtonBench reports symbolic accuracy (SA), RMSLE, and interactive discovery statistics over 324 tasks spanning 12 physics domains. UltraHorizon reports environment scores for long-horizon partially observable exploration, with trajectories designed to stress memory, planning, and tool use. ScienceAgentBench reports program validity, execution results, success rate, and cost over 102 tasks from 44 publications. Our main experiments use the first three benchmark families; ScienceAgentBench remains the natural workflow-execution target for the same hypothesis-state interface.

## Related Work

**Reasoning-time scaffolds for LLM agents.** Chain-of-thought and agentic prompting extend the computation performed at inference time. ReAct couples reasoning traces with actions (Yao et al. 2023a); Reflexion and Self-Refine use feedback to revise later attempts (Shinn et al. 2023; Madaan et al. 2023); Tree-of-Thoughts and LATS make the search tree explicit (Yao et al. 2023b; Zhou et al. 2024); CodeAct unifies tool use around executable Python actions (Wang et al. 2024). These methods are closest to MARS at the level of

execution, but they do not by themselves define what a scientific hypothesis object is. MARS uses the same broad agentic insight–interleave model calls with external computation–but makes the persistent unit a typed hypothesis state rather than a reasoning transcript.

**Self-optimizing language-model pipelines.** DSPy treats LM programs as optimizable computational graphs rather than hand-written prompt strings (Khattab et al. 2023). Self-Discover asks a model to compose reasoning structures before solving a task (Zhou et al. 2024). Voyager stores executable skills learned through interaction with Minecraft (Wang et al. 2023). These systems show that the right unit of adaptation may be a program, module, or reusable skill. MARS differs in two ways. First, its stored unit is tied to scientific evidence: contracts, measurements, residuals, and validators. Second, promotion is residual-conditioned; an operator is useful only if it closes typed holes or compresses recurrent failures across tasks.

**Program induction and library learning.** DreamCoder learns a domain language and neural search policy through a wake-sleep loop over solved and imagined programs (Ellis et al. 2021). Later library-learning work further explores how abstractions reduce search breadth and depth. This line is a major conceptual ancestor of MARS: both systems treat expertise as a changing language rather than a fixed prompt. The difference is that MARS operates in mixed scientific interfaces where the answer may be prose, a law, or a workflow, and where holes can often be closed by measurements rather than by full program synthesis.

**Symbolic regression and scientific law discovery.** Symbolic regression recovers compact formulas from data. AI-Feynman exploits physics-inspired structure such as symmetry and separability (Udrescu and Tegmark 2020); PySR provides a practical evolutionary symbolic-regression engine for scientific modeling (Cranmer 2023). NewtonBench moves beyond static formula fitting by requiring interactive rediscovery of shifted physical laws (Zheng et al. 2025). MARS borrows the insistence on executable hypotheses, but does not assume that every task reduces to equation recovery. Its coordinate probes are one operator family inside a broader hypothesis-state kernel.

**Causal and experimental-design perspectives.** Scientific hypotheses are not merely predictors; they make claims about stable relations under interventions, environments, or perturbations. Structural causal models formalize this distinction (Pearl 2009), invariant prediction uses stability across environments to identify causal structure (Peters, Buhlmann, and Meinshausen 2016), and Bayesian experimental design studies how to choose measurements that maximize information (Rainforth et al. 2023). MARS is not a causal discovery algorithm, but it adopts the same discipline: a hypothesis must state what evidence would support it, what transformation should preserve it, and what residual would falsify or refine it.

**Benchmarks for scientific agents.** DiscoveryBench, NewtonBench, UltraHorizon, and ScienceAgentBench expose different faces of the same problem: the agent must build an internal research object before it can answer. DiscoveryBench emphasizes data and metadata alignment (Majumder et al. 2024); NewtonBench emphasizes interactive law induction (Zheng et al. 2025); UltraHorizon emphasizes long-horizon memory and partial observability (Luo et al. 2025); ScienceAgentBench emphasizes executable scientific workflows (Chen et al. 2025). We use these benchmarks not as separate engineering targets but as probes of whether one hypothesis-state representation can support multiple scientific task interfaces.

# MARS

## Overview

MARS is a hypothesis-induction kernel around a small language model. The model is used to propose typed sketches, measurements, and repairs, but the state of the search is external and executable. The same loop is used for all benchmarks: compile a contract, generate artifacts, execute measurements, validate them, store residuals, and occasionally promote a residual pattern into a new operator. Operators do not pass only text to later calls; they update one typed context containing contracts, artifacts, evidence, residuals, and promoted operators.

## The MARSContext

For a task $x$, the context is

$$\mathcal{M}_x = (C, \mathcal{H}, \Theta, \mathcal{E}, \mathcal{R}, \mathcal{O}, h^\star), \tag{5}$$

where $C$ is the evidence contract, $\mathcal{H}$ is the pool of candidate artifacts, $\Theta$ stores closed typed holes, $\mathcal{E}$ stores executable evidence, $\mathcal{R}$ stores typed residuals, $\mathcal{O}$ is the active operator set, and $h^\star$ is the current best answer artifact. All operators read and write this object. This is the main architectural constraint: every stage must return a state update, not a private chain of thought.

Each operator $O_j \in \mathcal{O}$ has the same interface,

$$O_j(\mathcal{M}_x, x) \to \{(h, e, \rho, \Delta\mathcal{M})\}, \tag{6}$$

where $h$ is a proposed artifact, $e$ is executed evidence, $\rho$ is the typed residual, and $\Delta\mathcal{M}$ is the state update. This interface is what makes the system benchmark-independent. A tabular estimand, a symbolic coordinate probe, and a transition-rule inducer differ internally, but the kernel only sees artifact, evidence, residual, and update.

## End-to-End Trace

Figure 1 gives a concrete NewtonBench-style trajectory. The model does not guess the law directly. It proposes a typed sketch; execution detects residual curvature; the residual selects a coordinate probe; the probe closes the exponent and constant slots; and the held-out validator accepts the artifact. The failed first probe changes the typed state, not merely the next prompt.

| State | Update |
|---|---|
| Contract | symbolic law; slots: coordinate, exponent, constant, residual |
| Sketch | $y = c\phi(x)^p$ with holes $\{\phi, p, c\}$ |
| Probe | raw coordinate fit leaves `wrong_coordinate` residual |
| Closure | log-log probe sets $\phi(x) = x$, $p = 2.03 \to 2$, $c = 0.99 \to 1$ |
| Validate | held-out RMSLE below threshold; residual cleared; render $y = x^2$ |

Figure 1: A compact MARS trace. Typed residuals constrain later executable operators: here a coordinate residual activates a log-log probe and closes the law slots.

## Operators

**Contract compiler.** The contract compiler maps the input interface to required slots and local validators. It does not predict the gold answer. It predicts what any valid answer must make observable: answer type, scope, variables, measurement target, relation form, and rendering constraints.

**Typed-hole closure.** Instead of asking the model for a complete hypothesis, MARS asks for sketches with typed holes and then closes the holes by small executable probes whenever possible. For a tabular question, a hole may be an outcome role or a window operator. For a law-discovery task, it may be a coordinate chart or exponent. For a sequence task, it may be a state variable or transition clause. Closing a hole adds an assignment to $\Theta$ and reduces the search space for later holes.

**Executable estimands and coordinate probes.** The estimand operator creates statistical objects for tabular discovery: group contrasts, interaction effects, coefficient shifts, multi-item proportions, and time-window measurements. The coordinate-probe operator creates compact transformations for laws and rules: log-log fits, affine and polynomial lifts, ratios, finite differences, thresholds, and simple state-transition clauses. Both operators produce evidence objects rather than only prose rationales.

**Metamorphic verifier.** The verifier checks whether the artifact answers the requested question under transformations that should preserve the object: renaming irrelevant columns, holding answer form fixed, changing display order, perturbing nuisance values, or replaying held-out traces. These checks are label-free and therefore available before benchmark evaluation.

## Induction Algorithm

The score used in line 10 is the posterior-style objective from the preliminaries:

$$\begin{aligned}\text{score}(h) = \lambda_E E(h, x) + \lambda_C \,\text{cover}(h, C) \\ - \lambda_V V(h, C) - \lambda_K K(h).\end{aligned} \tag{7}$$

Here $V$ is metamorphic validation loss and $K$ is an artifact complexity penalty. The same score ranks natural-language discovery hypotheses, symbolic laws, and transition programs.

**Algorithm 1** MARS typed executable hypothesis induction

**Require:** task interface $x$, operator set $\mathcal{O}_0$, budget $B$
1: $C \leftarrow \textsc{CompileContract}(x)$
2: $\mathcal{M} \leftarrow (C, \emptyset, \emptyset, \emptyset, \emptyset, \mathcal{O}_0, \emptyset)$
3: **for** $t = 1$ to $B$ **do**
4: $\quad\mathcal{P} \leftarrow \emptyset$
5: $\quad$**for all** $O_j \in \mathcal{M}.\mathcal{O}$ **do**
6: $\quad\quad\mathcal{P} \leftarrow \mathcal{P} \cup O_j(\mathcal{M}, x)$
7: $\quad$**end for**
8: $\quad$**for all** $(h, e, \rho, \Delta\mathcal{M}) \in \mathcal{P}$ **do**
9: $\quad\quad$update $\mathcal{M}$ with $h, e, \rho, \Delta\mathcal{M}$
10: $\quad\quad$score $h$ by evidence, coverage, validation, and complexity
11: $\quad\quad$**if** $h$ improves $h^\star$ **then**
12: $\quad\quad\quad h^\star \leftarrow h$
13: $\quad\quad$**end if**
14: $\quad$**end for**
15: $\quad$**if** $h^\star$ satisfies $C$ and validation checks **then**
16: $\quad\quad$**return** $\textsc{Render}(h^\star, C)$
17: $\quad$**end if**
18: $\quad\mathcal{O}_{new} \leftarrow \textsc{CompileResiduals}(\mathcal{M}.\mathcal{R})$
19: $\quad$promote operators in $\mathcal{O}_{new}$ that pass cross-task checks
20: **end for**
21: **return** $\textsc{Render}(h^\star, C)$

## Residual-Conditioned Operator Promotion

The residual-extension step is not arbitrary code growth. A residual cluster $R = \{\rho_1, \ldots, \rho_m\}$ can propose an operator only if it identifies a repeatable missing transformation. Formally, a candidate operator $O'$ is promoted when

$$\Delta(O') = \mathbb{E}_{x \sim \mathcal{D}_{held}}[S(h^\star_{O'}(x)) - S(h^\star(x))] - \beta K(O') > 0, \tag{8}$$

and the improvement is not confined to the task that produced the residual. This rule is meant to prevent an ever-growing bag of benchmark-specific tricks: an operator must improve held-out tasks under the same contract and scoring interface while paying a complexity penalty.

## Why This Helps Small Models

MARS changes the role of the language model. The model does not need to hold the entire long-horizon proof in context or invent the final hypothesis in one sample. It repeatedly proposes local typed objects whose consequences can be executed. Each closed hole constrains later holes, each verifier rejects an invalid answer form before final scoring, and each residual becomes a structured training signal for the outer loop. The intended gain is therefore not more verbal reasoning, but a better external search geometry for weak models.

# Experimental Evaluation

## Benchmarks

We evaluate on three benchmark families that stress different interfaces but share the same underlying difficulty: the

agent must construct a hypothesis object before it can answer. DiscoveryBench provides real datasets, metadata, and natural-language discovery goals; the final answer is scored by Hypothesis Matching Score (HMS) and a consistency-HMS variant. NewtonBench provides interactive scientific-law discovery tasks in shifted physical worlds; we report symbolic accuracy over all tasks (SA-all) and over non-abstained answers (SA-answered). UltraHorizon provides long-horizon partially observable exploration tasks; we report the environment-compatible score and exact program-fit count for sequence-rule tasks. Table 2 gives the interfaces and metrics used in this draft.

## Baselines and protocol

The model core is either GPT-4o-mini or GPT-4o. We compare raw model/agent behavior to the same model wrapped by MARS whenever matched runs are available. Raw baselines include direct answering and simple tool-using agent loops; the paper treats ReAct, CodeAct, and self-debugging as reference families rather than as our contribution. The important controlled comparison is whether the same model improves when hypothesis generation is routed through contracts, executable artifacts, typed residuals, and the shared state.

We do not introduce benchmark-specific modules in the reported system. The operator set is shared: contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned promotion. Benchmark adapters only translate the native task interface into the generic contract representation and translate the final artifact back to the benchmark answer format. All reported scores use each benchmark's native metric; no cross-benchmark normalization is applied.

Published DiscoveryBench reference scores include 16.3 HMS for GPT-4-preview CodeGen, 15.4 HMS for GPT-4o ReAct, and 24.5 HMS for oracle-feedback Reflexion. The oracle setting is separated from autonomous non-oracle agents because it uses gold-derived feedback to revise the answer. Table 4 therefore reports the non-oracle comparison and the oracle upper reference side by side. NewtonBench reports 75.9 SA for a GPT-5 vanilla agent, 65.4 for Gemini-2.5-Pro, and 47.8 for o4-mini. UltraHorizon reports 14.33 average score for the best free-step LLM setting and 26.52 for humans under the paper protocol. We cite these numbers only as context; the MARS rows below are controlled mechanism tests, not a unified leaderboard.

## Main results

Table 5 separates controlled slices from broader audits for the GPT-4o-mini-core MARS system. The 30-task Discovery row tests the hard-bucket setting used during mechanism development; the full Discovery row is an audit over 239 DB-Real tasks. NewtonBench is reported on the 24 easy and 24 hard law-version slices. UltraHorizon is reported both as an all-environment two-seed audit and as a checked sequence-induction slice. The table is not a claim of full-benchmark SOTA. It tests whether the same hypothesis-state interface transfers across three task types. The pattern is consistent with the method design: DiscoveryBench is limited by answer-form induction and semantic rendering, NewtonBench by coverage and abstention, and UltraHorizon sequence induction benefits most from exact executable transition checks. The Discovery full audit remains below the controlled slice; it identifies residual families rather than supporting a leaderboard claim. The post-audit original–replication operator raises the full DB-Real audit from 13.6 to 20.4 HMS, with most of the gain concentrated in meta-regression and requirements-style table questions. In the published DB-Real context, this places the autonomous GPT-4o-mini-core system above the non-oracle GPT-4o ReAct and GPT-4-preview CodeGen references, but still below oracle-feedback Reflexion.

## Model-core comparison

Table 6 separates the language model core from the external hypothesis system on matched subsets. DiscoveryBench still shows a positive GPT-4o gap, consistent with semantic interpretation and answer phrasing being important. NewtonBench and UltraHorizon show a different regime: once the hypothesis is converted into executable probes, GPT-4o-mini can match or exceed the paired GPT-4o run under the same scaffold. The UltraHorizon rows come from paired controlled subsets, not from the checked 5/5 sequence slice; the point of the table is model-gap analysis, not headline scoring.

## Ablations

DiscoveryBench is the most sensitive benchmark for hypothesis quality because the final answer is a natural-language scientific relation. Table 7 reports the hard-bucket layer ablation used during mechanism development. The trend is monotone: evidence contracts alone are not enough, executable estimands improve measured hypothesis quality, role canonicalization improves both HMS and consistency, and the final role layer mainly increases consistency by preventing proxy columns from being treated as unrelated scientific variables.

Table 8 summarizes additional mechanism ablations across the three benchmark families. These are diagnostic matched-scope tests, not full leaderboard submissions. They show the intended pattern: typed execution helps most when the missing object is measurable. On DiscoveryBench, residual-conditioned study-design operators repair a full-audit residual family. On NewtonBench, coordinate charts turn a hard trigonometric slice from mostly failed or abstained answers into exact executable laws on the matched tasks. On UltraHorizon, typed slots and measurement compilation provide smaller but consistent gains on the all-environment audit.

## Residual-operator audit

The strongest novelty claim in MARS is the residual-to-operator pathway. We therefore report it separately from the main score table. Table 9 summarizes smoke and audit runs in which residual patterns were converted into executable rules or promoted operator families. These runs are not full official benchmark evaluations; they test whether residuals can change the hypothesis language in measurable ways.

Table 2: Benchmark interfaces used to evaluate whether one hypothesis-state kernel can operate across heterogeneous scientific reasoning tasks.

| Benchmark | Interface | Required artifact | Metric used here |
|---|---|---|---|
| DiscoveryBench | tables + metadata + goal | scientific hypothesis | HMS / consistency-HMS |
| NewtonBench | simulator + observations | symbolic law | SA-all / SA-answered |
| UltraHorizon | partially observable environment | transition program | score / exact program fit |

Table 3: Evaluation context. Published protocols and MARS runs are deliberately not merged into one numeric leaderboard because their task sets and judging surfaces differ. Published scores are discussed in text; this table records the protocol gap.

| Benchmark | Published protocol reference | MARS evaluation scope |
|---|---|---|
| DiscoveryBench | DB-Real autonomous and oracle-feedback agents | 30-task hard slice; full DB-Real audit |
| NewtonBench | 324 interactive law-discovery tasks | 24 easy and 24 hard law slices |
| UltraHorizon | three environments with repeated long-horizon runs | all-env two-seed audit; checked sequence slice |

Table 4: DiscoveryBench DB-Real context. Oracle feedback uses gold-derived revision signals, so it is not the same regime as autonomous non-oracle agents.

| System | Feedback | HMS |
|---|---|---|
| GPT-4o ReAct | none | 15.4 |
| GPT-4-preview CodeGen | none | 16.3 |
| MARS, 239-task audit | none | 20.4 |
| Reflexion | oracle | 24.5 |

Table 5: Evaluation results under the shared MARS operator set. Metrics are native to each benchmark family. "Slice" rows are controlled mechanism tests; "audit" rows exercise broader task coverage without claiming a full official leaderboard submission.

| Setting | Primary | Secondary |
|---|---|---|
| Discovery 30 slice | 40.5 HMS | 60.5 Cons. |
| Discovery 239 audit | 20.4 HMS | 27.1 Cons. |
| Newton easy | 83.3 SA-all | 95.2 SA-ans. |
| Newton hard | 41.7 SA-all | 83.3 SA-ans. |
| UH all-env easy | 49.2 score | 33.3 PI |
| UH all-env hard | 45.8 score | 33.3 PI |
| UH Seq checked | 5/5 exact | 80.0 score |

The audit is deliberately separated from the benchmark table because the evidence is mechanistic rather than comprehensive. In UltraHorizon sequence tasks, residual rules such as character-wise combination, positional maximum, and sorted multiset transformations close failures that a raw proposal loop missed. In the Newton smoke setting, a numeric residual kernel repairs an almost-correct power law on held-out points, not a separate full NewtonBench task split. In the DiscoveryBench outer-loop audit, promoted study-design operators infer original/replication arms, domain-code aliases, paired columns, binary complements, and contrastive renderings from table schemas. They move the full DB-Real audit from 13.6 to 20.4 HMS and the raw meta-regression subset from 0.7 to 24.2 HMS. The remaining plateau is equally important: the current mechanism can stabilize a better operator family, but does not yet guarantee continuing open-ended improvement across unrelated table families.

Table 6: Matched model comparison on identical task subsets where paired GPT-4o and GPT-4o-mini runs are available. UH rows are separate controlled subsets and should not be compared to the 5/5 checked sequence slice in Table 5.

| Setting | GPT-4o-mini | GPT-4o |
|---|---|---|
| DB stress | 64.29 | 69.84 |
| NB hard | 41.7 | 41.7 |
| UH raw | 0.0 | 6.67 |
| UH MARS | 50.0 | 43.33 |
| UH hard | 56.67 | 56.67 |

Table 7: DiscoveryBench layer ablation on the hard-bucket task set.

| System | N | HMS | Cons-HMS |
|---|---|---|---|
| MEK | 30 | 16.7 | 30.0 |
| MEK +estimands | 30 | 20.5 | 37.2 |
| +role canonicalization | 30 | 27.5 | 44.2 |
| +universal role layers | 30 | 40.5 | 60.5 |

## Failure analysis

The remaining errors fall into four recurring classes. *Rendering errors* occur when the system measures a valid relation but expresses it in a form that does not match the benchmark facet. These dominate DiscoveryBench. *Role errors* occur when the contract closes a plausible but wrong scientific role, often because several dataset columns proxy the same latent variable. *Coverage errors* occur when no active operator proposes the right coordinate chart, transition clause, or

Table 8: Mechanism ablations across benchmark families. Each row compares matched evaluation scopes and native metrics.

| Benchmark | Mechanism | Scope | Before→After |
|---|---|---|---|
| DiscoveryBench | hard-bucket role stack | 30 tasks | 16.7→40.5 HMS |
| DiscoveryBench | residual study-design operator | 239 tasks | 13.6→20.4 HMS |
| NewtonBench | self-induced coordinate charts | 4 hard tasks | 25.0→75.0 SA-all |
| NewtonBench | final epistemic checks | 24 easy tasks | 79.2→83.3 SA-all |
| UltraHorizon | typed slots + measurements | 6 easy env runs | 46.7→50.8 score |
| UltraHorizon | sequence residual operators | checked hard seq | 2/5→5/5 exact |

Table 9: Residual-operator audit. Source and promotion columns distinguish where the residual was observed from where the candidate operator was checked.

| Slice | Operator | Source | Promotion eval. | Before→After |
|---|---|---|---|---|
| UH Seq | string rules | train traces | held-out rules | 2/5→5/5 |
| Newton smoke | power law | residual fit | held-out points | 0.85→1.00 |
| Discovery | study-design roles | failed tasks | DB-Real full audit | 13.6→20.4 |
| Discovery raw meta-reg. | paired-slot renderer | failed tasks | 50-task subset | 0.7→24.2 |
| Mean norm. | G0→G1 | outer loop | suite audit | .789→.801 |

estimand family. These dominate NewtonBench abstentions. *Assembly errors* occur when the task requires a large multi-file workflow; this is the main reason ScienceAgentBench is treated as a target interface rather than as a headline result in this draft.

The 239-task Discovery audit makes these errors concrete. After residual operator promotion, the system is strong when the task is close to executable slot closure or paired study-design measurement: 73.6 HMS on the NLS raw subset, 36.0 HMS on WorldBank indicator tasks, 24.5 HMS on meta-regression, 24.2 HMS on raw meta-regression, 26.7 HMS on requirements-engineering questions, and 23.7 HMS on archaeology tasks. The same audit exposes negative transfer and uncovered families: plant introduction pathways fall to 6.3 HMS, NLS SES falls to 7.4 HMS, and one WorldBank education-GDP subset remains at 0.0 HMS. The dominant remaining DiscoveryBench failures are denominator alignment, multi-file assembly, and semantic rendering, not arithmetic measurement.

This failure analysis is also the practical value of the MARS state. In a transcript-only agent, failures are often just bad final answers. In MARS, the failure is attached to the missing typed object: a role, a coordinate, a validator, a renderer, or an operator. That makes later improvement measurable without changing the benchmark-specific code path.

## Limitations

MARS is a hypothesis-induction architecture, not evidence that small language models can autonomously solve scientific discovery in general. The results in this draft are controlled slices, not full official benchmark submissions. They are useful because they expose the mechanism under matched conditions, but they should not be read as a completed state-of-the-art claim.

The strongest results occur when the task exposes measurements or traces that can close typed holes. When the needed operator is absent, the kernel cannot recover the right hypothesis by ranking alone. This makes operator coverage the main remaining technical bottleneck. DiscoveryBench additionally exposes a rendering problem: the system can measure a scientifically plausible relation but express it in a form that misses the benchmark's target facet. NewtonBench errors are concentrated in abstentions, compile failures, and missing coordinate charts rather than in confidently accepted wrong laws. UltraHorizon sequence induction is strong in the checked setting, but the full benchmark requires more environments and repeated runs because single trajectories have high variance. ScienceAgentBench is excluded from the headline evaluation because full scientific workflow execution requires a stronger multi-file assembler.

The residual-operator audit is also limited. It demonstrates that residuals can produce reusable operators on held-out slices, but it does not yet prove unbounded autonomous growth. The current system has a plateau after the first outer-loop improvement. A stronger version must learn broader operator families from residuals while retaining the same contract, validation, and complexity controls.

**DiscoveryBench positioning.** The correct reading of the DiscoveryBench result is not "full SOTA." The strong claim is that a small-model system without gold feedback can exceed the published non-oracle GPT-4o ReAct and GPT-4-preview CodeGen references on the DB-Real audit, while oracle-feedback Reflexion remains a higher revision-aided reference. This distinction matters because the proposed method targets autonomous hypothesis-state construction rather than access to answer-level feedback.

## Conclusion

We presented MARS, a typed executable hypothesis-induction system for scientific reasoning with small language

models. The main claim is structural: weak models benefit when hypothesis generation is moved from free-form prose into an external state of typed artifacts, executable measurements, validators, and residuals. Across data-driven discovery, symbolic law induction, and long-horizon sequence induction, the same kernel improves the effective reasoning of GPT-4o-mini on controlled slices and sometimes reduces the gap to GPT-4o. The experiments do not show that scaffolding eliminates model capability gaps or that the system has already reached full-benchmark SOTA. They show where external structure helps: when the system can decompose a hypothesis into measurable slots, use execution to close them, and turn recurrent typed residuals into better operators. This suggests a concrete path for small-model scientific agents: grow the hypothesis language itself, guided by residuals, rather than only sampling more reasoning traces from a fixed language.

## References


Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. DiscoveryBench: Towards data-driven discovery with large language models. arXiv preprint arXiv:2407.01725, 2024.

Tianshi Zheng, Kelvin Kiu-Wai Tam, Newt Hue-Nam K. Nguyen, Baixuan Xu, Zhaowei Wang, Jiayang Cheng, Hong Ting Tsang, Weiqi Wang, Jiaxin Bai, Tianqing Fang, Yangqiu Song, Ginny Y. Wong, and Simon See. NewtonBench: Benchmarking generalizable scientific law discovery in LLM agents. arXiv preprint arXiv:2510.07172, 2025.

Haotian Luo, Huaisong Zhang, Xuelin Zhang, Haoyu Wang, Zeyu Qin, Wenjie Lu, Guozheng Ma, Haiying He, Yingsha Xie, Qiyang Zhou, Zixuan Hu, Hongze Mi, Yibo Wang, Naiqiang Tan, Hong Chen, Yi R. Fung, Chun Yuan, and Li Shen. UltraHorizon: Benchmarking agent capabilities in ultra long-horizon scenarios. arXiv preprint arXiv:2509.21766, 2025.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *ICLR*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*, 2024.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *ICML*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. arXiv preprint arXiv:2310.03714, 2023.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-Discover: Large language models self-compose reasoning structures. arXiv preprint arXiv:2402.03620, 2024.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. In *PLDI*, 2021.

Silviu-Marian Udrescu and Max Tegmark. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16), 2020.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv preprint arXiv:2305.01582, 2023.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Peter Buhlmann, and Nicolai Meinshausen. Causal inference by using invariant prediction: Identification and confidence intervals. *Journal of the Royal Statistical Society: Series B*, 78(5):947–1012, 2016.

Tom Rainforth, Adam Foster, Desi R. Ivanova, and Freddie Bickford Smith. Modern Bayesian experimental design. arXiv preprint arXiv:2302.14545, 2023.